

Transclive — System Documentation

Project: Transclive

Type: Real-time audio transcription and summarization web application

Frontend: Next.js 16 + React 19 + Tailwind CSS 4 + shadcn/ui

Backend: Node.js + Express 5 + TypeScript

Database: PostgreSQL via Prisma 7

Real-time transcription: Deepgram Nova-3 over WebSocket

Authentication: JWT in an HTTP-only cookie

Summary generation: OpenAI-compatible chat-completions endpoint using `gpt-4o-mini`

1. Project Overview

Transclive is a web application that records microphone audio in the browser, streams the audio to the backend in real time, forwards the audio to Deepgram for speech-to-text processing, displays interim and final transcript segments to the user, persists completed transcriptions in PostgreSQL, and generates a persistent AI summary after a recording is saved.

The system has two independently deployable applications:

- **Client:** Next.js application providing authentication, recording controls, live transcript display, history, search, transcript detail, editing, deletion, copying, and JSON export.
- **Server:** Express application serving REST APIs and a WebSocket endpoint. It owns authentication, Deepgram integration, transcription persistence, summary generation, and database access.

The current implementation also contains a provider-agnostic local extractive summary service in `summary.service.ts`; however, the active `/api/transcriptions/:id/summary` controller currently uses the configured AI API directly.

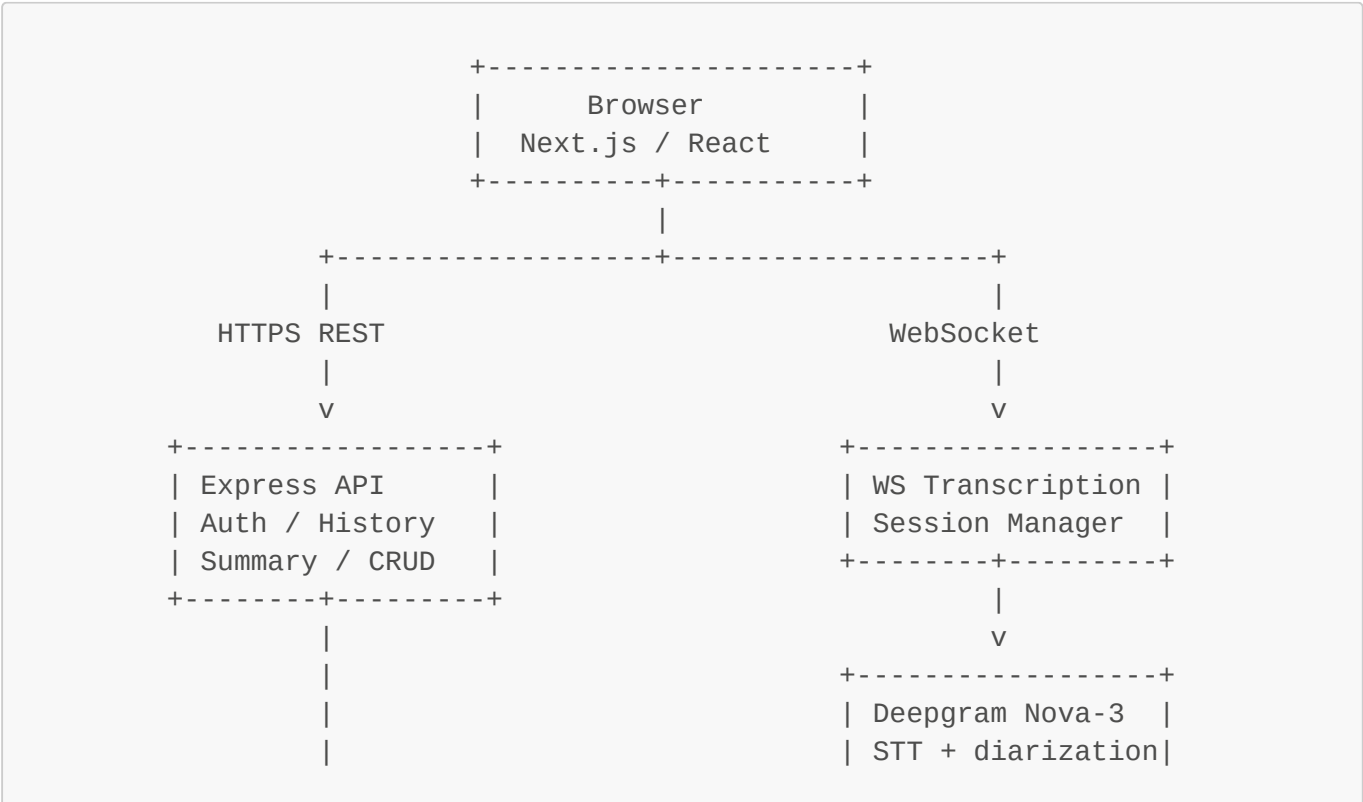
2. Objectives

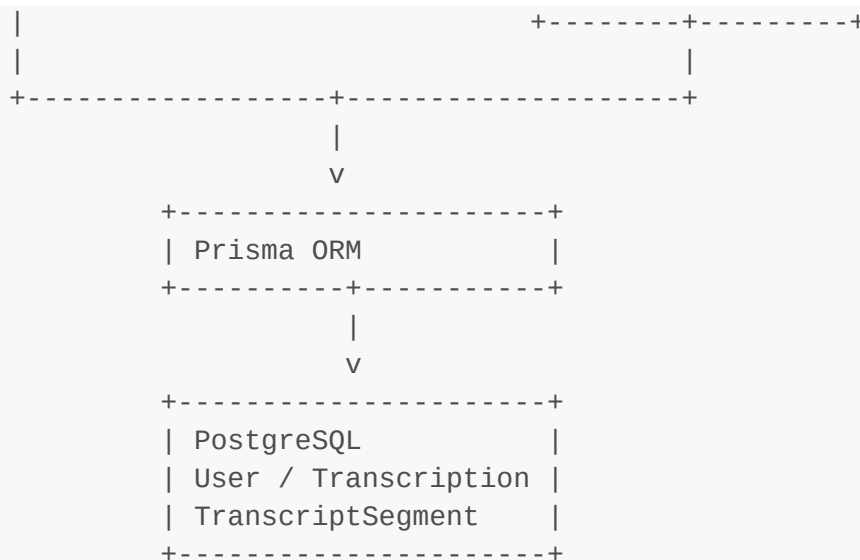
1. Capture microphone audio from a browser.
 2. Stream audio continuously without waiting for the recording to finish.
 3. Provide low-latency interim and final transcription results.
 4. Support configurable transcription languages.
 5. Detect speakers through Deepgram diarization.
 6. Persist completed transcripts and timestamped segments.
 7. Allow users to search, rename, view, copy, export, and delete saved transcriptions.
 8. Automatically initiate summary generation after a transcription is successfully saved.
 9. Persist generated title and summary.
 10. Handle authentication, validation, API errors, WebSocket errors, and summary failures.
-

3. Technology Stack

Layer	Technology
UI	Next.js 16.3, React 19.2
Styling	Tailwind CSS 4, shadcn/ui
Icons	lucide-react
API server	Express 5.2
Language	TypeScript
Real-time transport	WebSocket (ws)
Speech-to-text	Deepgram SDK, Nova-3
Database	PostgreSQL
ORM	Prisma 7
Authentication	JWT + HTTP-only cookie
Password hashing	Argon2
Validation	Zod
Security middleware	Helmet, CORS
Logging	Morgan
Rate limiting dependency	express-rate-limit
AI summary	OpenAI-compatible chat completions endpoint

4. High-Level Architecture





Summary path:

PostgreSQL transcript -> Express summary controller -> AI API -> PostgreSQL

5. Main Modules

5.1 Authentication Module

The authentication module provides:

- Registration
- Login
- Logout
- Current-user lookup
- Password hashing with Argon2
- JWT creation and verification
- HTTP-only cookie storage
- Protected REST routes
- Protected WebSocket connections

JWT payload:

```
{
  userId: <user-id>
}
```

Token lifetime is seven days.

5.2 Real-Time Recording Module

The browser requests microphone access using:

```
navigator.mediaDevices.getUserMedia({ audio: true })
```

The audio is encoded using **MediaRecorder**. The implementation prefers:

1. **audio/webm;codecs=opus**
2. **audio/webm**
3. **audio/mp4**

Chunks are emitted every approximately 250 ms and sent as binary WebSocket messages.

5.3 WebSocket Transcription Module

Endpoint:

```
/ws/transcribe
```

The WebSocket connection is authenticated using the **auth_token** cookie.

Client control messages:

```
{"type": "start", "language": "en-US"}
```

```
{"type": "stop"}
```

```
{"type": "retry_save"}
```

Server messages include:

```
ready  
transcript  
stopping  
transcription_saved  
utterance_end  
deepgram_closed  
error
```

5.4 Deepgram Module

The backend opens a Deepgram Nova-3 connection with:

- `smart_format`
- `interim_results`
- `punctuate`
- `diarize`
- `endpointing`
- `utterance_end_ms`

The backend parses Deepgram results into normalized transcript segments.

Each segment contains:

- text
- start time
- end time
- confidence
- dominant speaker
- word-level information

5.5 Persistence Module

A completed recording is persisted as:

```
Transcription
|
+-- TranscriptSegment
+-- TranscriptSegment
+-- ...
```

Segments are ordered by their `start` timestamp.

5.6 Summary Module

When a transcription is saved, the client receives:

```
transcription_saved
```

The client immediately calls:

```
POST /api/transcriptions/:id/summary
```

The backend:

1. Authenticates the user.
2. Loads the transcription and segments.
3. Concatenates segment text.
4. Changes `summaryStatus` to `PENDING`.
5. Calls the configured AI endpoint.
6. Requests JSON containing `title` and `summary`.
7. Validates the returned JSON.
8. Saves the generated title and summary.
9. Changes `summaryStatus` to `READY`.

If any step fails, `summaryStatus` becomes `FAILED`.

6. Detailed Recording and Summary Flow

This is the most important workflow in the current implementation.

```
User
|
| Click "Start recording"
v
Client
|
| WebSocket connect
v
Backend
|
| Verify auth_token JWT
v
WebSocket session
|
| start(language)
v
Deepgram
|
| ready
v
Client
|
| MediaRecorder.start(250)
|
| binary audio chunks
v
WebSocket backend
|
| sendMedia()
v
Deepgram Nova-3
|
| interim/final Results
```

```
v
Backend parser
|
| transcript message
v
Client UI
|
| final segments accumulated
|
| User clicks "Stop recording"
v
MediaRecorder.stop()
|
| final chunk emitted
v
Client
|
| { type: "stop" }
v
Backend
|
| close Deepgram connection
v
Deepgram close event
|
| saveTranscription()
v
PostgreSQL
|
| transcription + segments saved
v
Backend
|
| transcription_saved(id)
v
Client
|
| POST /api/transcriptions/:id/summary
v
Backend summary controller
|
| load transcript
| status = PENDING
v
AI API
|
| title + summary JSON
v
Backend
|
| persist title + summary
| status = READY
v
PostgreSQL
```

```
|  
v  
Client  
|  
| show "Recording saved"  
| display summary on history/detail page
```

Important implementation detail

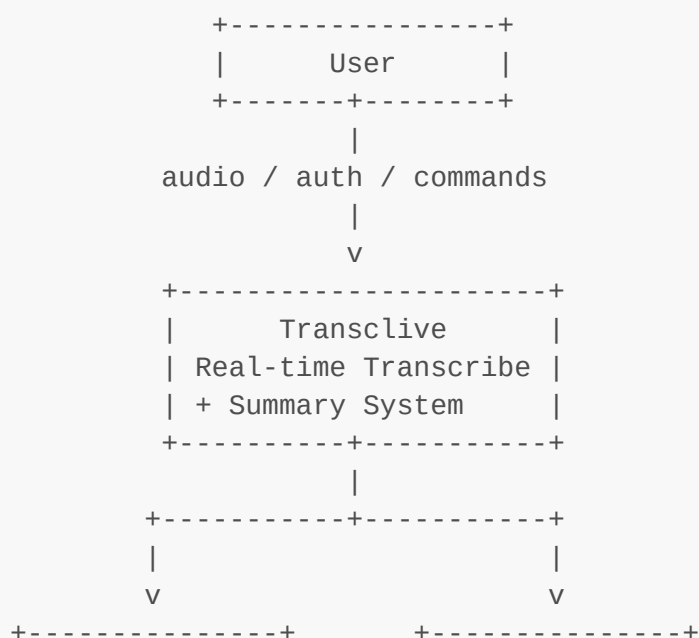
The summary is **not generated directly inside the Deepgram close handler**.

The sequence is:

```
Deepgram closes  
↓  
saveTranscription()  
↓  
transcription_saved WebSocket event  
↓  
client receives saved ID  
↓  
client calls POST /summary  
↓  
AI summary is generated
```

This separation prevents summary generation from being coupled directly to the audio-stream shutdown operation.

7. DFD — Level 0 / Context Diagram





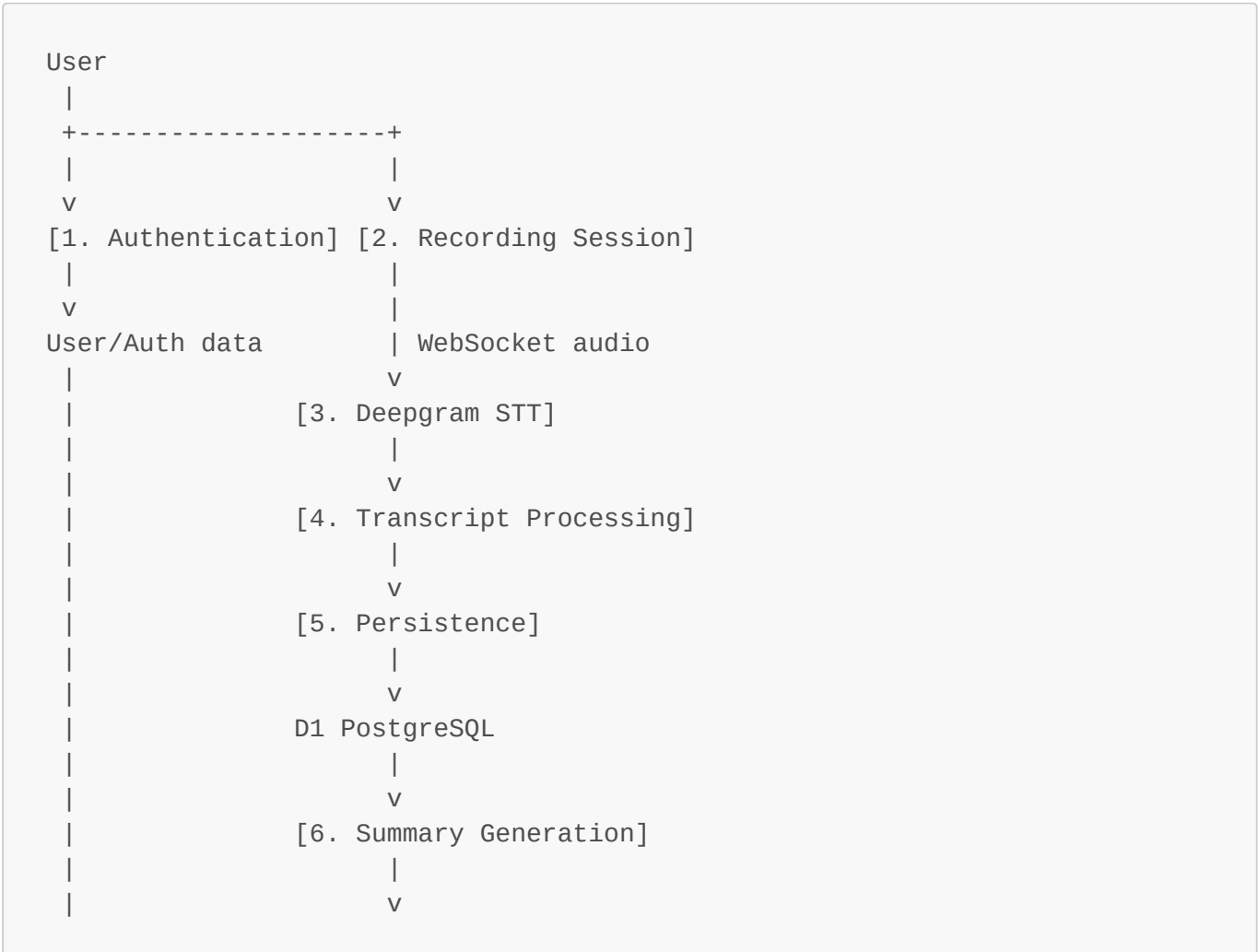
External entities

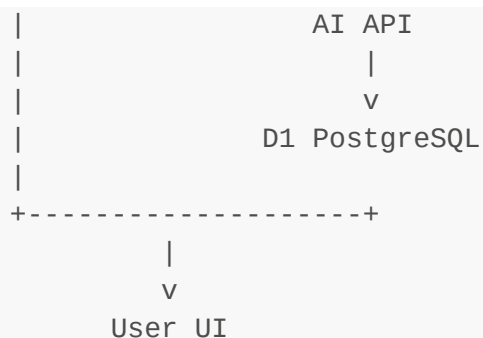
- User
- Deepgram
- AI API
- PostgreSQL is treated as the system's persistent data store

Main system process

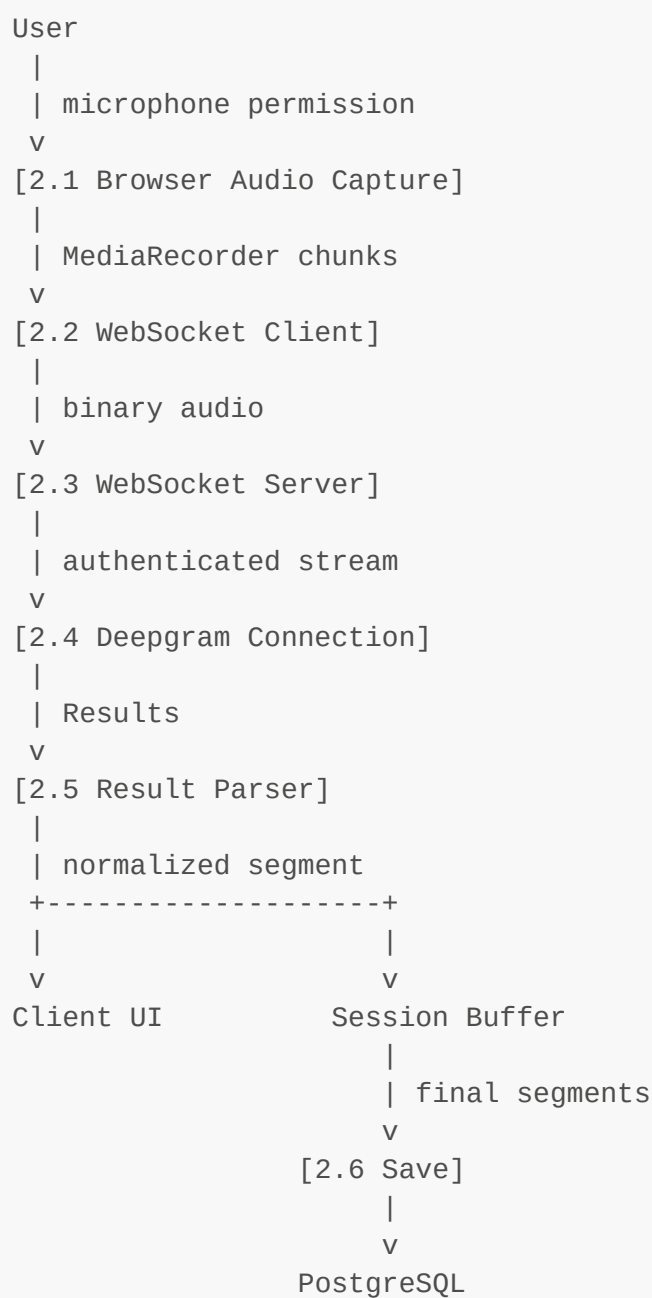
- Transclive

8. DFD — Level 1

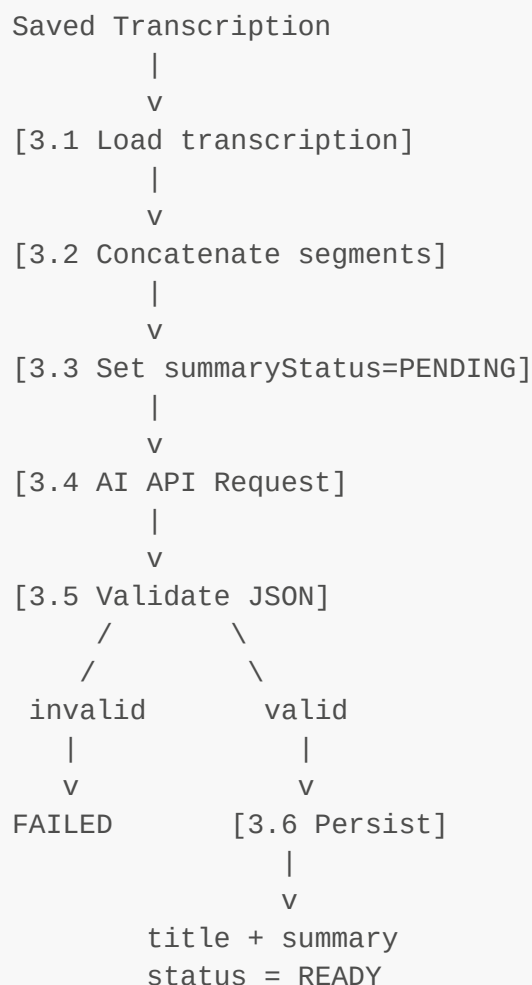




9. DFD — Level 2: Recording

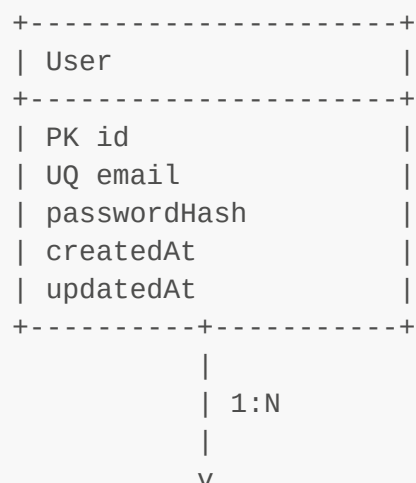


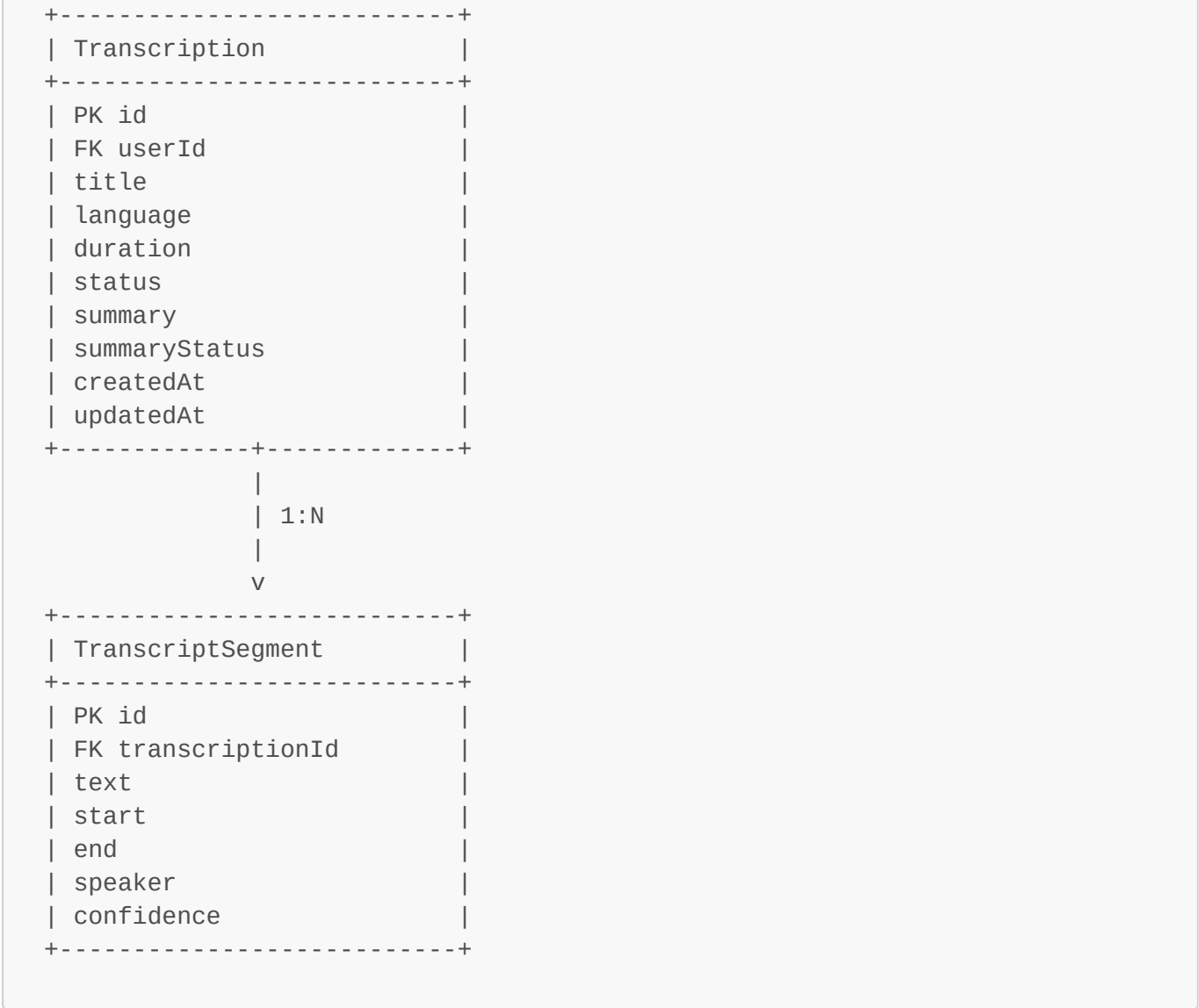
10. DFD — Level 2: Summary Generation



11. Database Design

Entity Relationship Model





User

Field	Type	Description
id	String	CUID primary key
email	String	Unique login identifier
passwordHash	String	Argon2 password hash
createdAt	DateTime	Account creation
updatedAt	DateTime	Last update

Transcription

Field	Type	Description
id	String	CUID primary key
userId	String	Owner
title	String?	User/AI generated title

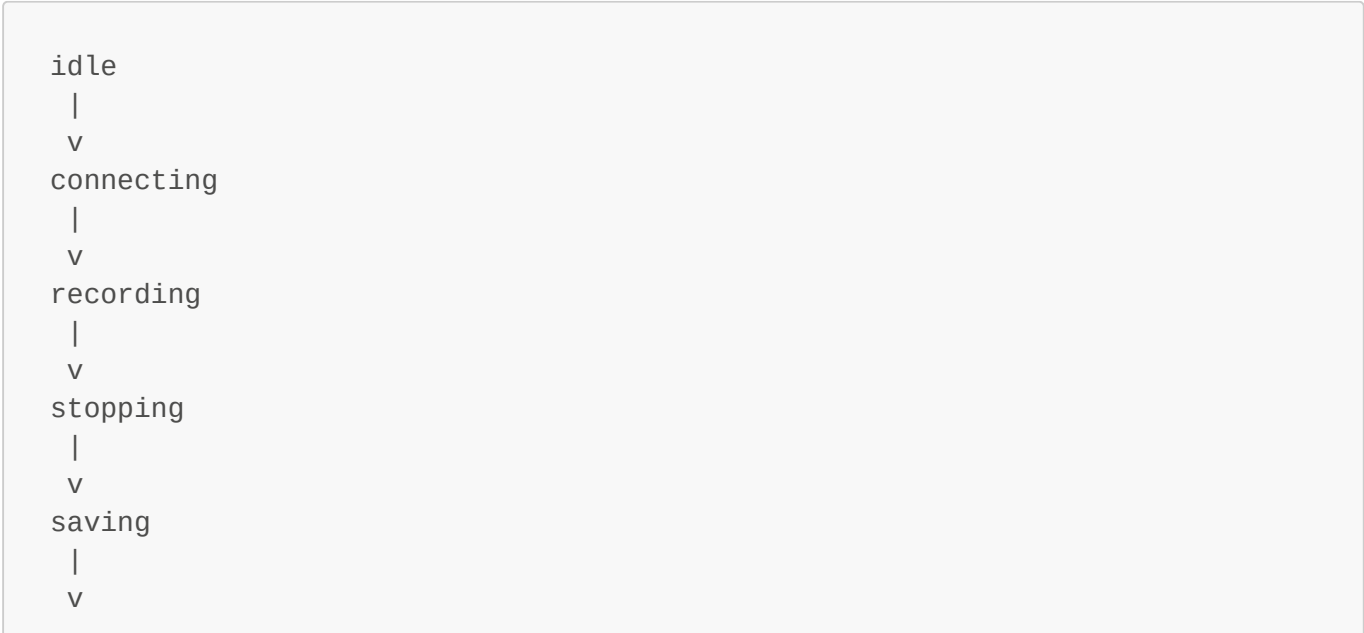
Field	Type	Description
language	String	Transcription language
duration	Float?	Recording duration
status	Enum	RECORDING / COMPLETED / FAILED
summary	String?	Generated summary
summaryStatus	Enum	NONE / PENDING / READY / FAILED
createdAt	DateTime	Creation time
updatedAt	DateTime	Last update

TranscriptSegment

Field	Type	Description
id	String	CUID primary key
transcriptionId	String	Parent transcription
text	String	Segment text
start	Float	Start timestamp
end	Float	End timestamp
speaker	Int?	Diarized speaker
confidence	Float?	Recognition confidence

12. State Machines

Client Recording State



```
graph TD
    summarizing --> plus[+-----+]
    plus --> ready
    plus --> summary_failed[summary_failed]
    summary_failed --> retry
```

Failure branch:

```
saving -> save_failed -> retry_save
```

Summary State

```
graph TD
    NONE --> request_summary[request summary]
    request_summary --> PENDING
    PENDING --> READY
    PENDING --> FAILED
    FAILED --> retry
    retry --> PENDING
```

13. REST API

Authentication

POST </api/auth/register>

Creates an account and sets the authentication cookie.

Request:

```
{
  "email": "user@example.com",
```

```
"password": "password123"
}
```

POST `/api/auth/login`

Authenticates an existing user.

POST `/api/auth/logout`

Clears the authentication cookie.

GET `/api/auth/me`

Returns the currently authenticated user.

Transcriptions

GET `/api/transcriptions`

Returns the authenticated user's transcriptions.

Optional query parameter:

```
?search=<query>
```

Search applies to:

- title
- transcript segment text

GET `/api/transcriptions/:id`

Returns a transcription and its ordered segments.

PATCH `/api/transcriptions/:id`

Renames a transcription.

Request:

```
{
  "title": "Meeting Notes"
}
```

DELETE `/api/transcriptions/:id`

Deletes a transcription owned by the authenticated user.

Because the database uses cascading deletion, child transcript segments are removed with the transcription.

POST `/api/transcriptions/:id/summary`

Generates and persists a summary and AI-generated title.

14. WebSocket API

Endpoint:

```
/ws/transcribe
```

Authentication

The server extracts:

```
auth_token
```

from the HTTP cookie and verifies the JWT before accepting the session.

Client → Server

Start

```
{
  "type": "start",
  "language": "en-US"
}
```

Stop

```
{
  "type": "stop"
}
```

Retry Save

```
{  
  "type": "retry_save"  
}
```

Binary messages contain microphone audio.

Server → Client

Ready

```
{  
  "type": "ready"  
}
```

Transcript

```
{  
  "type": "transcript",  
  "segment": {},  
  "isFinal": true,  
  "speechFinal": true  
}
```

Saving

```
{  
  "type": "stopping"  
}
```

Saved

```
{  
  "type": "transcription_saved",  
  "transcription": {  
    "id": "..."  
  }  
}
```

Error

```
{  
  "type": "error",  
  "message": "...",  
  "code": "..."  
}
```

15. Authentication and Security

Password security

Passwords are hashed with Argon2 before database storage.

JWT

JWTs contain the authenticated user's ID and expire after seven days.

Cookie

The JWT is stored in:

```
auth_token
```

with:

- **HttpOnly**
- **Secure** in production
- **SameSite** configured according to environment
- seven-day expiration

REST authorization

Protected routes use **requireAuth**.

WebSocket authorization

The WebSocket handshake independently verifies the same JWT cookie.

Request validation

Registration and login requests use Zod schemas.

HTTP hardening

Helmet is enabled.

CORS

Allowed browser origins are controlled through `ALLOWED_ORIGINS`.

16. Error Handling

The backend uses a centralized `AppError` / `errorHandler` pattern.

Operational errors can expose their message in production, while unexpected errors are hidden behind:

```
Something went wrong on our end.
```

Important error cases include:

- Authentication required
- Invalid/expired authentication token
- User not found
- Transcription not found
- Empty transcript
- Missing AI API key
- AI API failure
- Invalid AI JSON
- Failed transcription persistence
- Deepgram failure

17. Frontend Routes

Route	Purpose
/	Landing/root page
/login	Login
/register	Registration
/app	Real-time recording dashboard
/history	Searchable transcription history
/history/[id]	Transcript detail

The dashboard shell provides navigation between recording and history.

18. Frontend Features

Recording Page

- Language selector
- Connection status
- Start/stop recording
- Live waveform
- Interim transcript
- Final transcript segments
- Speaker labels
- Error messages
- Save status
- Summary generation status
- Summary retry

History Page

- Search by title/content
- List saved recordings
- Delete recording
- Open recording detail

Search is debounced by approximately 300 ms.

Transcript Detail

- Rename title
- Delete
- Copy transcript
- Export JSON
- Generate summary
- Display summary
- Display timestamped transcript segments
- Display speaker labels

19. Sequence Diagram — Authentication

```
User -> Client: Submit email/password
Client -> Express: POST /api/auth/login
Express -> Prisma: Find user
Prisma -> Express: User
Express -> Argon2: Verify password
Argon2 -> Express: Valid
Express -> JWT: Create token
JWT -> Express: Token
Express -> Client: Set HTTP-only cookie
Client -> User: Logged in
```

20. Sequence Diagram — Live Transcription

```
User -> Client: Start recording
Client -> WebSocket: Connect
WebSocket -> JWT: Verify cookie
JWT -> WebSocket: userId
Client -> WebSocket: start(language)
WebSocket -> Deepgram: Open Nova-3
Deepgram -> WebSocket: ready
WebSocket -> Client: ready

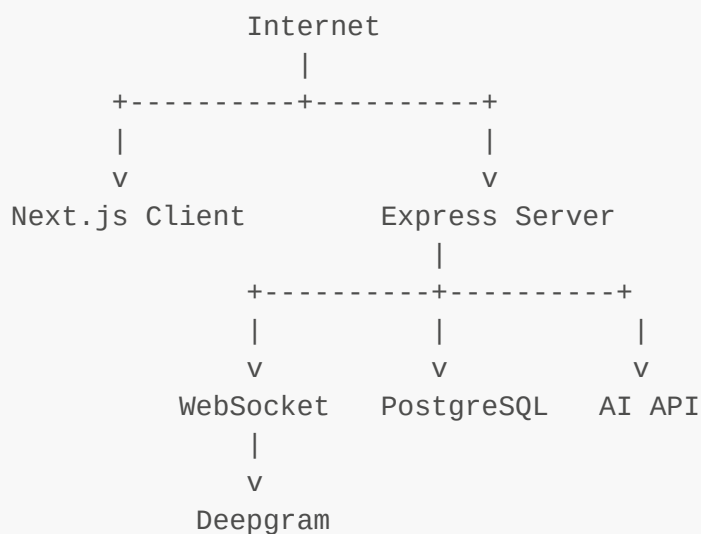
loop while recording
  Client -> WebSocket: Binary audio chunk
  WebSocket -> Deepgram: sendMedia()
  Deepgram -> WebSocket: Results
  WebSocket -> Parser: Normalize result
  Parser -> WebSocket: TranscriptSegment
  WebSocket -> Client: transcript
  Client -> UI: Render segment
end
```

21. Sequence Diagram — Stop and Summary

```
User -> Client: Stop recording
Client -> MediaRecorder: stop()
MediaRecorder -> Client: final audio chunk
Client -> WebSocket: stop
WebSocket -> Deepgram: close()
Deepgram -> WebSocket: close event
WebSocket -> Persistence: createTranscription()
Persistence -> PostgreSQL: INSERT transcription
Persistence -> PostgreSQL: INSERT segments
PostgreSQL -> Persistence: saved
WebSocket -> Client: transcription_saved(id)

Client -> Express: POST /api/transcriptions/:id/summary
Express -> PostgreSQL: Load transcription + segments
Express -> PostgreSQL: summaryStatus=PENDING
Express -> AI API: Generate title + summary
AI API -> Express: JSON
Express -> PostgreSQL: Save title + summary
Express -> PostgreSQL: summaryStatus=READY
Express -> Client: Updated transcription
Client -> UI: Display summary
```

22. Deployment Architecture



Environment variables used by the backend include:

```
PORT
DATABASE_URL
AUTH_SECRET
DEEPGRAM_API_KEY
AI_API_KEY
ALLOWED_ORIGINS
NODE_ENV
```

The client uses:

```
NEXT_PUBLIC_API_URL
```

for API/WebSocket connectivity.

23. Current Implementation Notes

Summary implementation

There are currently two summary implementations in the server source:

1. `summary.service.ts` contains a local extractive summarizer.
2. `createSummary` in `transcription.controller.ts` currently performs the active summary operation through the external AI API.

The second implementation is the one used by the current route.

Port configuration

The backend's default port in `server.ts` is:

```
8000
```

The frontend uses `NEXT_PUBLIC_API_URL` when configured. Its fallback values in different parts of the current source are not completely consistent (`8000` in the generic API helper and `8016` in the WebSocket hook). Deployment should therefore explicitly set `NEXT_PUBLIC_API_URL`.

Migration history

The Prisma migrations show an evolution from a single JSON/text transcript representation to normalized `TranscriptSegment` records and persistent summary status. The current `schema.prisma` should be treated as the authoritative current model.

24. Non-Functional Requirements

Performance

- Audio is streamed in small chunks.
- Interim transcription is delivered without waiting for recording completion.
- Transcript segments are stored incrementally in memory during a session and persisted at completion.
- History search is debounced on the frontend.

Reliability

- Deepgram connection errors are surfaced to the client.
- Database save failure has a retry mechanism.
- Summary failure is persisted as `FAILED` and can be retried.
- Authentication is independently checked for REST and WebSocket access.

Security

- Passwords are not stored in plaintext.
- JWT is not exposed through localStorage.
- Browser authentication uses an HTTP-only cookie.
- API requests include credentials.
- CORS and Helmet are configured on the server.

Maintainability

The backend separates:

- routes

- controllers
 - services
 - middleware
 - validators
 - database access
 - external integrations
-

25. Testing Recommendations

The implementation should be tested at the following levels.

Unit Tests

- JWT creation/verification
- Password validation
- Deepgram result parsing
- Dominant speaker calculation
- Summary sentence scoring
- API validation schemas

Integration Tests

- Register/login/logout
- Protected route access
- Create and retrieve transcription
- Rename/delete transcription
- Search transcription
- Summary generation
- Summary failure state

WebSocket Tests

- Authentication failure
- Start session
- Binary audio forwarding
- Interim results
- Final results
- Stop session
- Persistence after Deepgram close
- Retry save

End-to-End Tests

1. Register.
2. Login.
3. Start recording.
4. Speak.

5. Verify interim text.
6. Verify final text.
7. Stop.
8. Verify persistence.
9. Verify automatic summary generation.
10. Open history.
11. Open transcript detail.
12. Rename/copy/export/delete.

26. Future Enhancements

Potential improvements:

- Background/asynchronous summary generation
- Queue-based summary jobs
- Summary progress via WebSocket
- Multiple summary formats
- Export to TXT, Markdown, PDF
- Audio recording persistence
- Playback synchronized with transcript timestamps
- Full word-level transcript navigation
- Better speaker naming
- User-configurable AI models
- Rate limiting on summary generation
- Automated retry with exponential backoff
- Structured logging
- Observability and metrics
- Automated test suite
- CI/CD pipeline
- API documentation using OpenAPI

27. Conclusion

Transclive currently implements the core architecture required for a real-time transcription platform:

```
Browser microphone
  ↓
MediaRecorder
  ↓
Authenticated WebSocket
  ↓
Deepgram Nova-3
  ↓
Live transcript
```

```
graph TD; A[ ] --> B[Persisted transcription]; B --> C[AI-generated title + summary]; C --> D[Searchable history];
```

↓
Persisted transcription
↓
AI-generated title + summary
↓
Searchable history

The most significant architectural characteristic is the separation between **real-time transcription**, **persistence**, and **summary generation**. The recording can therefore be finalized and saved before the summary request is executed, while summary failures can be retried independently.